

MINI PROJECT - DAY 7

Title: Order Analytics Pipeline

STEP 1 - Linux Setup

mkdir order_pipeline

cd order_pipeline

touch orders.csv analysis.py

Add the 10-row orders dataset from today's

SQL section into orders.csv.

```
-rwxrwxrwx 1 prashant prashant 0 Mar 18 2026 summary.py
drwxrwxrwx 1 prashant prashant 4.0K Mar 18 2026 order_pipeline
prashant@DESKTOP-B9KE2N7: /mnt/d/kode-x notes/python/python_exercise's/day6_project$ cd order_pipeline/
prashant@DESKTOP-B9KE2N7: /mnt/d/kode-x notes/python/python_exercise's/day6_project/order_pipeline$ touch orders.csv analysis.py
prashant@DESKTOP-B9KE2N7: /mnt/d/kode-x notes/python/python_exercise's/day6_project/order_pipeline$ ls -lrth
total 0
-rwxrwxrwx 1 prashant prashant 0 Mar 18 2026 orders.csv
-rwxrwxrwx 1 prashant prashant 0 Mar 18 2026 analysis.py
prashant@DESKTOP-B9KE2N7: /mnt/d/kode-x notes/python/python_exercise's/day6_project/order_pipeline$ cat > orders.csv << EOF
order_id,customer_name,product,category,quantity,unit_price,city,order_date
1,Rahul,Laptop,Electronics,2,55000,Pune,2024-01-05
2,Priya,Phone,Electronics,3,20000,Mumbai,2024-01-07
3,Amit,Desk,Furniture,1,12000,Pune,2024-01-10
4,Neha,Laptop,Electronics,1,55000,Delhi,2024-01-12
5,Karan,Chair,Furniture,4,4500,Bangalore,2024-01-15
6,Sneha,Phone,Electronics,2,20000,Pune,2024-01-18
7,Rohan,Bookshelf,Furniture,2,8000,Mumbai,2024-01-20
8,Divya,Tablet,Electronics,3,15000,Delhi,2024-01-22
9,Meera,Desk,Furniture,1,12000,Pune,2024-01-25
10,Arjun,Laptop,Electronics,1,55000,Bangalore,2024-01-28
EOF
prashant@DESKTOP-B9KE2N7: /mnt/d/kode-x notes/python/python_exercise's/day6_project/order_pipeline$ cat orders.csv | column -t -s,
order_id customer_name product category quantity unit_price city order_date
1 Rahul Laptop Electronics 2 55000 Pune 2024-01-05
2 Priya Phone Electronics 3 20000 Mumbai 2024-01-07
3 Amit Desk Furniture 1 12000 Pune 2024-01-10
4 Neha Laptop Electronics 1 55000 Delhi 2024-01-12
5 Karan Chair Furniture 4 4500 Bangalore 2024-01-15
6 Sneha Phone Electronics 2 20000 Pune 2024-01-18
7 Rohan Bookshelf Furniture 2 8000 Mumbai 2024-01-20
8 Divya Tablet Electronics 3 15000 Delhi 2024-01-22
9 Meera Desk Furniture 1 12000 Pune 2024-01-25
10 Arjun Laptop Electronics 1 55000 Bangalore 2024-01-28
prashant@DESKTOP-B9KE2N7: /mnt/d/kode-x notes/python/python_exercise's/day6_project/order_pipeline$ |
```

STEP 2 - Python Task

Function 1: read_orders(filename)

- Reads orders.csv

- Skips header

- Returns a list of dictionaries

one dict per row, keys from header

```

#Function 1: read_orders(filename)

def read_orders(filename):
    with open(filename, "r", newline="") as file:
        data = file.readlines()
        header = data[0].strip().split(',')

        order = [dict(zip(header,line.strip().split(','))) for line in data[1:]]
    return order

# Function 2: total_revenue(orders)

def total_revenue(orders):
    return sum(int(data['quantity']) * int(data['unit_price']) for data in orders)

# Function 3: revenue_by_category(orders)

def revenue_by_category(orders):
    dict_category = {}
    for i in orders:
        if i['category'] not in dict_category:
            dict_category[i['category']] = 0

        dict_category[i['category']] += int(i['quantity']) * int(i['unit_price'])
    return dict_category

```

```

# Function 4: top_customers(orders, n=3)

def top_customers(orders, n=3):
    top_customers = {}
    for i in orders:
        if i['customer_name'] not in top_customers:
            top_customers[i['customer_name']] = 0

        top_customers[i['customer_name']] = int(i['quantity']) * int(i['unit_price'])
        sorted_cus = sorted(top_customers.items(), key=lambda x: [x[1]],reverse=True)
    return sorted_cus

# Function 5: city_order_count(orders)

def city_order_count(orders):
    city_count = {}
    for i in orders:
        if i['city'] not in city_count:
            city_count[i['city']] = 0

        city_count[i['city']] += 1
    return city_count

```

```

def print_report(orders, *args, **kwargs):
    print("\n SUMMARY REPORT")
    print("===="*5)

    print(f"Total Revenue: {total_revenue(orders)}")
    print("\nRevenue By Category:")
    for key, value in revenue_by_category(orders).items():
        print(key, ':', value)

    n = kwargs.get("top_n", 3)
    print(f"\nTop {n} Customers:")
    for key,value in top_customers(orders,n):
        print(key, ':', value)

    print("\nOrders By City:")
    for key,value in city_order_count(orders).items():
        print(key, ':', value)

orders = read_orders("orders.csv")
print_report(orders,top_n=3)

```

```

SUMMARY REPORT
=====
Total Revenue: 423000

Revenue By Category:
Electronics : 365000
Furniture : 58000

Top 3 Customers:
Rahul : 110000
Priya : 60000
Neha : 55000

Orders By City:
Pune : 4
Mumbai : 2
Delhi : 2
Bangalore : 2

```

STEP 3 - SQL Verification

Run SQL Exercise 5 from today's SQL section.

Compare output with your Python report.

Both must match.

QUERY

SQL Exercise 5 - Combined Challenge

Write a single query that produces this summary:

- Category name
- Total orders
- Total quantity sold
- Total revenue
- Average order value (rounded to 0 decimals)

Show only categories with total revenue above 50000.

Order by total revenue descending.

```
CREATE TABLE TODAY_ORDERS1(  
    ORDER_ID NUMBER,  
    CUSTOMER_NAME VARCHAR2(20),  
    PRODUCT VARCHAR2(20),  
    CATEGORY VARCHAR2(20),  
    QUANTITY NUMBER,  
    UNIT_PRICE NUMBER,  
    CITY VARCHAR2(20),  
    ORDER_DATE DATE  
);  
  
INSERT INTO TODAY_ORDERS1 VALUES (1, 'Rahul', 'Laptop', 'Electronics', 2, 55000, 'Pune', DATE '2024-01-05');  
INSERT INTO TODAY_ORDERS1 VALUES (2, 'Priya', 'Phone', 'Electronics', 3, 20000, 'Mumbai', DATE '2024-01-07');  
INSERT INTO TODAY_ORDERS1 VALUES (3, 'Amit', 'Desk', 'Furniture', 1, 12000, 'Pune', DATE '2024-01-10');  
INSERT INTO TODAY_ORDERS1 VALUES (4, 'Neha', 'Laptop', 'Electronics', 1, 55000, 'Delhi', DATE '2024-01-12');  
INSERT INTO TODAY_ORDERS1 VALUES (5, 'Karan', 'Chair', 'Furniture', 4, 4500, 'Bangalore', DATE '2024-01-15');  
INSERT INTO TODAY_ORDERS1 VALUES (6, 'Sneha', 'Phone', 'Electronics', 2, 20000, 'Pune', DATE '2024-01-18');  
INSERT INTO TODAY_ORDERS1 VALUES (7, 'Rohan', 'Bookshelf', 'Furniture', 2, 8000, 'Mumbai', DATE '2024-01-20');  
INSERT INTO TODAY_ORDERS1 VALUES (8, 'Divya', 'Tablet', 'Electronics', 3, 15000, 'Delhi', DATE '2024-01-22');  
INSERT INTO TODAY_ORDERS1 VALUES (9, 'Meera', 'Desk', 'Furniture', 1, 12000, 'Pune', DATE '2024-01-25');  
INSERT INTO TODAY_ORDERS1 VALUES (10, 'Arjun', 'Laptop', 'Electronics', 1, 55000, 'Bangalore', DATE '2024-01-28');
```

```
SELECT CATEGORY,  
    COUNT(*) AS TOTAL_ORDERS,  
    SUM(QUANTITY) AS TOTAL_QUANTITY,  
    SUM(QUANTITY * UNIT_PRICE) AS TOTAL_REVENUE,  
    ROUND(AVG(QUANTITY * UNIT_PRICE), 0) AS AVERAGE  
FROM TODAY_ORDERS1  
GROUP BY CATEGORY  
HAVING SUM(QUANTITY * UNIT_PRICE) > 50000  
ORDER BY TOTAL_REVENUE DESC;
```

Output :-

Query result Script output DBMS output Explain Plan SQL history					
  Download Execution time: 0.007 seconds					
	CATEGORY	TOTAL_ORDERS	TOTAL_QUANTITY	TOTAL_REVENUE	AVERAGE
1	Electronics	6	12	365000	60833
2	Furniture	4	8	58000	14500

STEP 4 - Linux Archiving

Once analysis.py runs successfully:

1. Create a results folder.

```
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/python/python_exercise's/day6_project$ mkdir -p results
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/python/python_exercise's/day6_project$ ls -lrth
total 0
-rwxrwxrwx 1 prashant prashant 225 Mar 18 2026 employees.csv
-rwxrwxrwx 1 prashant prashant 225 Mar 18 2026 sorted_by_dept.csv
-rwxrwxrwx 1 prashant prashant 0 Mar 18 2026 summary.py
drwxrwxrwx 1 prashant prashant 4.0K Mar 18 2026 order_pipeline
drwxrwxrwx 1 prashant prashant 4.0K Mar 18 2026 results
```

2. Copy analysis.py and orders.csv into results.

```
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/python/python_exercise's/day6_project/order_pipeline$ cp analysis.py orders.csv results/
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/python/python_exercise's/day6_project/order_pipeline$ ls -lrth
total 20K
-rwxrwxrwx 1 prashant prashant 587 Mar 18 2026 orders.csv
-rwxrwxrwx 1 prashant prashant 2.2K Mar 18 2026 analysis.py
-rwxrwxrwx 1 prashant prashant 13K Mar 18 2026 mini_projects.ipynb
drwxrwxrwx 1 prashant prashant 4.0K Mar 18 2026 results
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/python/python_exercise's/day6_project/order_pipeline$ cd results/
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/python/python_exercise's/day6_project/order_pipeline/results$ ls -lrth
total 4.0K
-rwxrwxrwx 1 prashant prashant 2.2K Mar 18 2026 analysis.py
-rwxrwxrwx 1 prashant prashant 587 Mar 18 2026 orders.csv
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/python/python_exercise's/day6_project/order_pipeline/results$ |
```

3. Archive the results folder:

results.tar.gz

```
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/python/python_exercise's/day6_project/order_pipeline$ tar -czvf results.tar.gz results/
results/
results/analysis.py
results/orders.csv
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/python/python_exercise's/day6_project/order_pipeline$
```

4. Verify contents of the archive using tar -tvf.

```
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/python/python_exercise's/day6_project/order_pipeline$ tar -tvf results.tar.gz
drwxrwxrwx prashant/prashant 0 2026-03-18 13:35 results/
-rwxrwxrwx prashant/prashant 2151 2026-03-18 13:30 results/analysis.py
-rwxrwxrwx prashant/prashant 587 2026-03-18 13:30 results/orders.csv
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/python/python_exercise's/day6_project/order_pipeline$ s|
```

5. Set correct permissions:

chmod 644 on all .csv files

chmod 755 on analysis.py